

Supplementary Information: A Memristive Neural Decoder for Cryogenic Fault-Tolerant Quantum Error Correction

Victor Yon,^{1, 2, 3, 4, *} Frédéric Marcotte,^{5, 2, 3, 4, †} Pierre-Antoine Mouny,^{1, 2, 3, 4} Gebremedhin A. Dagnew,⁵ Bohdan Kulchytskyi,⁵ Sophie Rochette,¹ Yann Beilliard,^{5, 2, 3, 4} Dominique Drouin,^{1, 2, 3, 4} and Pooya Ronagh^{1, 6, 7, 8, ‡}

¹*Irréversible Inc., Sherbrooke, Québec, Canada*

²*Institut Interdisciplinaire d'Innovation Technologique (3IT),
Université de Sherbrooke, Sherbrooke, Québec, Canada*

³*Laboratoire Nanotechnologies Nanosystèmes (LN2 - IRL 3463) - CNRS,
Université de Sherbrooke, Sherbrooke, Québec, Canada*

⁴*Institut quantique (IQ), Université de Sherbrooke, Sherbrooke, Québec, Canada*

⁵*1QB Information Technologies (1QBit), Vancouver, British Columbia, Canada*

⁶*Institute for Quantum Computing, University of Waterloo, Waterloo, Ontario, Canada*

⁷*Department of Physics & Astronomy, University of Waterloo, Waterloo, Ontario, Canada*

⁸*Perimeter Institute for Theoretical Physics, Waterloo, Ontario, Canada*

(Dated: February 24, 2025)

S1. SIMULATION OF QUANTUM STABILIZER CIRCUITS

Here we provide some background on the simulation of surface codes and the error model used to generate the dataset for the neural decoder's training and testing (the dataset is available for download [1]). Our experiment is based on the distance-3 rotated surface code. This surface code corresponds to a 17-qubit system in which there are 9 data qubits (circles filled in white in Fig.2(a) of the main manuscript) and 8 ancilla qubits (circles filled in black) corresponding to the 8 stabilizer generators $S_1^X, S_2^X, S_3^X, S_4^X, S_1^Z, S_2^Z, S_3^Z$, and S_4^Z .

In our study, we rely on Stim [2] to generate the circuits shown in Fig.2 required to simulate the rotated surface code, PyMatching [3] for decoding with minimum-weight perfect matching in order to benchmark the relative performance of the recurrent neural network (RNN) decoder, and a custom-built glue code to integrate Stim and PyMatching. While writing this paper, the need for the glue code was eliminated due to recent updates in PyMatching2.

We simulate the memory- X rotated surface code, where preparation and final measurements are done in the X -basis. In addition, we conduct this simulation under circuit level noise, which means the following:

- with a noise probability of p , each two-qubit gate is followed by a two-qubit Pauli error drawn uniformly and independently from $\{I, X, Y, Z\}^{\otimes 2} \setminus \{I \otimes I\}$;
- with a probability of $2p/3$, the preparation of the $|0\rangle$ state is replaced by the state $|1\rangle = X|0\rangle$. Similarly, with a probability of $2p/3$, the preparation of the $|+\rangle$ state is replaced by the state $|-\rangle = Z|+\rangle$;
- with a probability of $2p/3$, any single-qubit measurement has its outcome flipped; and
- with a probability of p , each idling-qubit location is followed by a Pauli error drawn uniformly and independently from the stabilizer set $\{X, Y, Z\}$.

A Stim circuit is first initialized with a probability p for a given code distance and a number of rounds. This circuit is then repeatedly called to generate samples. The steps taken during circuit generation for performing the parity-check rounds shown in Fig.2(b) and (c) of the main manuscript are detailed as follows:

1. At the onset of the quantum error correction (QEC) cycle, the data qubits are initialized in the $|+\rangle$ state (for memory- X experiments), while ancilla qubits are set to the $|0\rangle$ state. These are replaced with the states $|+\rangle$ and $|1\rangle$, respectively, with a probability of $2p/3$, accounting for preparation errors.
2. Before the beginning of every parity check round, each data qubit is depolarized with a probability of p due to idling.

* These authors contributed equally to this work.; Corresponding author: victor.yon@usherbrooke.ca

‡ Corresponding author: pooya@irreversible.tech

† These authors contributed equally to this work.

3. To perform the parity-check circuits depicted in Fig.2(b) and (c), Hadamard gates are applied to the X -syndrome qubits, converting the $|0\rangle$ states to $|+\rangle$ states and the $|1\rangle$ states to $|-\rangle$ states, followed by single-qubit depolarizing noise with a probability of p .
4. Next, four CNOT cycles are executed, and each cycle is succeeded by two-qubit depolarizing noise with a probability of p , following the configuration outlined in Fig.2(b) and (c).
5. Hadamard gates are applied to the X -syndrome ancilla qubits, followed by depolarizing noise with a probability of p .
6. With a measurement error probability of $2p/3$, all ancilla qubits are measured in the Z -basis, after which they are reset to the $|0\rangle$ state.
7. To simulate preparation errors for the next QEC round, bit flips are applied with a probability of $2p/3$ on the ancilla qubits, along with idling, and steps 2 to 7 are repeated to conduct repeated syndrome measurement cycles.
8. Finally, the data qubits are measured in the X -basis with a measurement error a probability of $2p/3$. The data qubit measured implicitly define one final round of computational-basis-type stabilizers (in this case X stabilizers) inferred via the corresponding parity-check matrix. Since this is a classical process and is not subject to the circuit-level noise defined during a typical stabilizer round, the syndromes extracted in this round are also referred to as “perfect syndromes”. The final state of the encoded logical qubit is then determined.

The first stabilizer measurement round is the encoding round, which creates an entangled logical qubit. In the absence of an error, the encoding results in a logical $|+\rangle$ state since the data qubits are initialized in a logical $|+\rangle$ state. Measuring a $|-\rangle$ logical state in the end thus indicates a logical error, and the goal of a decoder is to predict such events.

To generate samples for training the RNN, we first separate the X and Z syndromes, focusing only on the X syndromes. Next, we divide the X -syndrome measurements into separate time steps, corresponding to each syndrome extraction round. The training label represents the logical bit-flip value, where 0 indicates that the logical state corresponding to the given syndromes has remained unchanged, and 1 indicates that the logical state has flipped. The training, validation, and testing samples are generated using noise probabilities between $p = 0.00001$ and $p = 0.01$, along with 3 rounds of stabilizer measurement and then a reset.

A key preprocessing step before feeding measurements to the decoder for analysis is to take the differences between consecutive measurement outcomes. Syndromes indicate the presence or absence, as well as the type, of errors that have occurred to the physical qubits. The RNN outputs a prediction of 0 or 1, representing the absence or presence of a logical error, respectively. The corresponding recovery operator is thus given by $(Z_L)^y$, where y is the binary output of the RNN.

With depolarizing noise, a weighted matching graph can be built and used for decoding, and in the noise capacity model (i.e., with perfect syndrome measurements) neural decoding can at best emulate minimum-weight perfect matching (MWPM) decoder. However, in presence of measurement errors, and at the circuit level noise model, a neural decoder can capture non-trivial patterns of X and Z errors propagation and therefore outperform MWPM [4]. Neural decoders can be trained with data from simulated noise channels (like in this paper, depolarizing noise channels) or using data collected from experiments. If done using experimental data, the decoder may capture the realistic noise affecting the quantum circuits without the need for sophisticated characterization of the noise channel. In practice, both steps may be applied, i.e., pre-training using a simulated circuit level noise model and further training using experimental data.

S2. OTHER HARDWARE NON-IDEALITIES

Analog-to-Digital and Digital-to-Analog Converter Range and Resolution

To simulate the behaviour of analog-to-digital converters (ADCs) and digital-to-analog converters (DACs) during inference on the crossbar arrays of memristors, a rounding function is applied,

$$f(x) = \begin{cases} -b & x < -b \\ \text{round}\left(\frac{x}{2b}n\right)\frac{2b}{n} & -b \leq x \leq b \\ b & x > b \end{cases}, \quad (\text{S1})$$

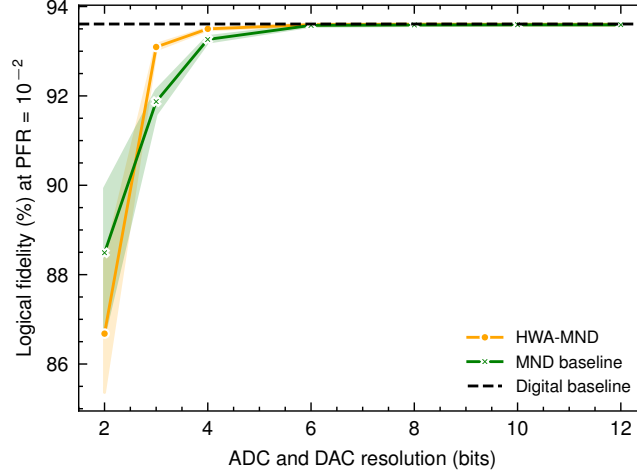


Figure S1. Impact of the analog-to-digital converter (ADC) and digital-to-analog converter (DAC) resolution on the recurrent neural network (RNN) decoder’s test fidelity. The physical fault rate (PFR) is set to 10^{-2} . The test fidelity is obtained without including reading variability or defective devices, in order to assess the impact of resolution alone. At an 8-bit resolution, that is, 256 quantization steps between the boundary values -6 and 6 , the fidelity approaches the digital baseline fidelity (using a 32-bit floating point accuracy). The error range is obtained based on 10 independent training rounds.

where x represents the ADC and DAC input, its range is given by $[-b, b]$, and n is the number of quantization steps, for example, 256 for 8-bit resolution. The round function rounds the floating-point value to the closest integer. Based on the typical matrix–vector multiplication (MVM) output values, we fix the ADC range to $b = 6$. The DAC range is fixed to $b = 1$ to match with the sigmoid output range. In ADCs and DACs, there is a trade-off between resolution and acquisition frequency, that is, a lower resolution yields a higher acquisition frequency. An application where inference time needs to be minimized, such as with the neural decoder, requires a high acquisition frequency. From Fig. S1, we can see that an 8-bit resolution is appropriate to reach the highest acquisition frequency without substantially degrading the RNN decoder’s fidelity. Therefore, we simulate ADC and DAC discretization during the testing of a memristive neural decoder (memristive neural decoder (MND)), but no re-training strategy seems necessary to maintain the decoder’s fidelity.

Reading Variability

Every analog electronic component induces a small reading variability. Analog-to-digital converters and digital-to-analog converters [5]; differential amplifiers; and memristors [6] involved in MVM operations are no exception. Based on experimental characterizations [7], we estimate that the dominant source of reading variability should come from memristors. To reduce the computational complexity, we simplify the simulation by considering every source of reading variability as one Gaussian random value added at the output of each row of memristors. As a conservative estimate, we chose a Gaussian distribution with a standard deviation equal to 1% of the maximum logical value (defined as b in the previous section).

The effect of different reading noise on the neural decoder’s fidelity is illustrated in Fig. S2. This simulation result suggests that a 1% reading noise is not detrimental to our decoding application. However the performance drops quickly when the value increases above this limit. We can observe that the random dropconnect applied with our hardware-aware memristive neural decoder (hardware-aware (HWA)-MND) re-training method also increases the robustness of the model against high reading variability. The device-specific memristive neural decoder (device-specific (DS)-MND) re-training method is more sensitive to perturbations that are not related to the stuck-at fault devices; this is visible by the faster fidelity drop in Fig. S2 when the output noise is increased in comparison to the HWA-MND method. This reading variability study shows that realistic noise values do not impact the performance enough to require a re-training method addressing specifically this issue. However, it is worth noting that a dropconnect strategy, applied in addition to the DS-MND method, could mitigate the negative effect of reading variability.

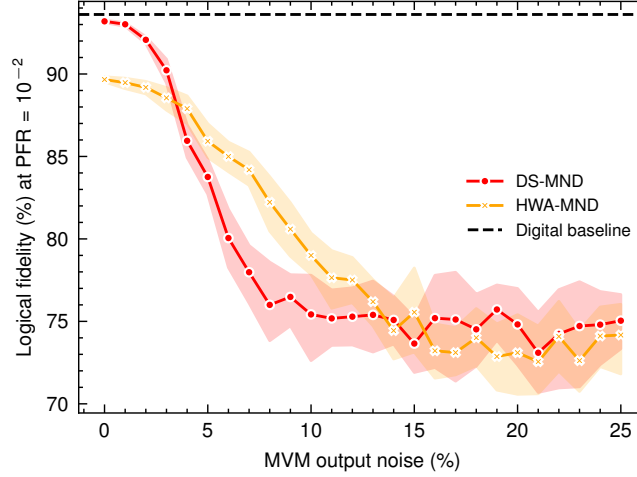


Figure S2. Impact of the matrix–vector multiplication (MVM) output noise on the recurrent neural network (RNN) decoder’s test fidelity. The physical fault rate (PFR) is set to 10^{-2} . A random Gaussian value is added to the output of each memristor row to account for the expected reading noise produced by the electronic components and the memristors. The percentage is relative to the maximum logical output value (set to 6 in our simulations). All other non-idealities are kept as defined in Table S2.

S3. EXPERIMENTS ON TiO_x -BASED RESISTIVE MEMORY DEVICES

The studied TiO_x -based resistive memory devices are characterized by a Keysight Technologies B1500A semiconductor device parameter analyzer that has a 200 million samples/second waveform generator / fast measurement unit (WGFMU) using a Lake Shore CPX-VF probe station at room temperature. For all measurements, the electrodes at the bottom are grounded and the signals are applied to the electrodes at the top.

Conductance State Programming

In our study, the resistance of memristors is programmed using a closed-loop read–write–verify algorithm [7, 8]. This algorithm allows the programming of a memristor to a target resistance by applying successive read and write pulses. Initially, a device’s resistance is read using a read pulse (having a 0.2 V amplitude and a 1 μs pulse width). If this measured resistance is larger than the target within a 1% tolerance, a positive write pulse is applied (~ 1 V amplitude and 200 ns pulse width); if it is lower, a negative pulse is applied. A read pulse is then applied to check the new resistance state, which starts a new cycle. The pulse amplitude is increased after each consecutive pulse of the same polarity by a constant step in order to converge to the target resistance. Using this read–write–verify algorithm for various target levels of resistance, we can perform multilevel programming as shown in Fig.4(a) in the main manuscript.

Programming Variability Characterization

The programming variability model is based on experimental measurement conducted on TiO_x -based resistive memory devices. As shown in Fig.4(a) in the main manuscript, multilevel programming is performed from the high-conductance states to the low-conductance states. To assess the cycle-to-cycle programming variability (shown in Fig.4(b)), we successively perform multilevel programming from the low-conductance states to the high-conductance states 10 times. For each state and at each multilevel programming pass, the conductance is read 20 times and the mean value is kept. This process is repeated for 10 memristors to include device-to-device variability in the programming error model. The histogram of the mean programmed values for a conductance state is fitted by a Gaussian distribution and its standard deviation is extracted (also depicted in Fig.4(b)). The fit of the standard deviation σ corresponds to the programming variability which depends on the conductance of the programmed state.

Conductance State Retention

The retention measurements presented in Fig. S3 are performed at room temperature for 8 hours for 5 conductance states. After the programming of a conductance state, a 1 ms-wide pulse with a 200 mV amplitude is applied every 5 minutes to measure the retention. The retention of the conductance states is measured successively. The 5 states demonstrate non-volatility over the eight-hour retention test. The conductance drift after 8 hours with respect to the initial conductance state can be fitted by a second-order polynomial function. Considering the small drift for this memristor technology, we have not considered this behaviour in the simulations presented in this paper.

S4. NEURAL NETWORK TRAINING METHODS

Implementation of Training and Inference

The training of the RNN always begins with a classical deep learning [9] optimization process, using PyTorch [10] with no consideration for hardware non-idealities (see Fig.5(a) in the main manuscript). The metaparameters used during this step are given in Table S1. In the case of hardware-aware memristive neural decoder (HWA-MND) and device-specific memristive neural decoder (DS-MND) re-training, the RNN parameters (weights and biases) are loaded from an earlier digital training, and one additional training epoch is run to adapt the model to newly introduced hardware constraints. Every other metaparameter remains unchanged during re-training.

Metaparameters	Values
Input size	4 rounds of 4 binary syndromes
Hidden layer size	32
Activation function	ReLU
Batch size	16
Loss	Cross-entropy
Optimizer	Adam [11]
Learning rate	0.001
Training epochs	4 (+1 for HWA and DS methods)

Table S1. Model and training metaparameters.

The RNN re-training and inference simulations are performed using the IBM Analog Hardware Acceleration Kit [12] v0.9.0. This Python library is used to simulate the computations executed on a crossbar array of memristors and the

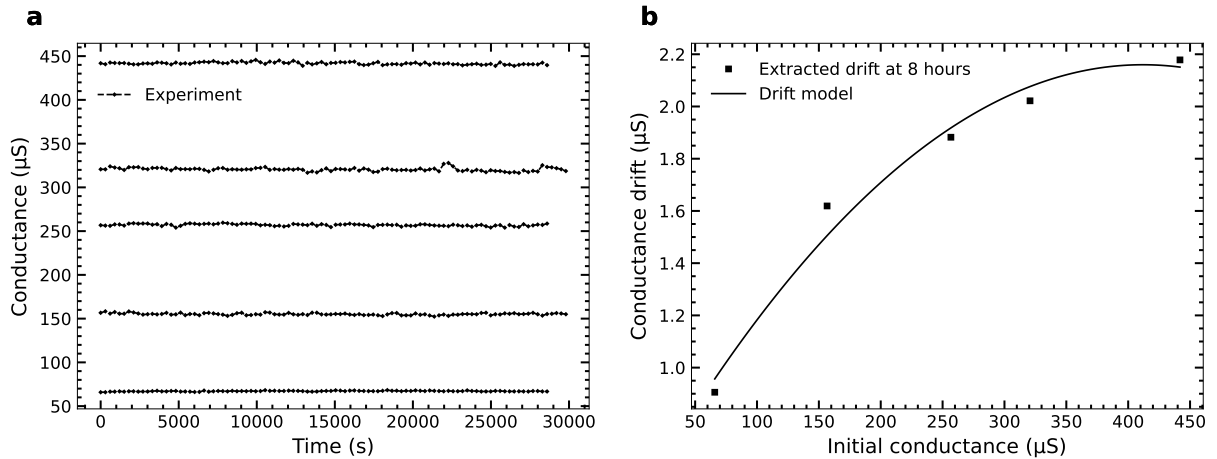


Figure S3. TiO_x-based resistive memory device retention study. (a) Retention of 5 conductance states between 65 μS and 440 μS. No conductance drift is noticeable. (b) Conductance drift of the 5 conductance states after eight-hour-retention measurements. The drift with respect to the initial conductance is fitted by a second-order polynomial function.

Hardware constraints and non-idealities	Simulated values
Weight clipping	2.5σ of the Gaussian distribution of each layer’s weights’ values
DAC resolution	8 bits
ADC resolution	8 bits
DAC max logical value	1
ADC max logical value	6
MVM output noise	Gaussian noise added with a standard deviation equal to 1 % of the maximum logical value (6)
Max memristor resistance	15 000 Ω
Min memristor resistance	5000 Ω
Memristor stuck-at-fault rate	10 %
Programming noise’s polynomial fit	$y = -x^3(2.40 \times 10^{-5}) + x^2(1.15 \times 10^{-2}) - x(7.07 \times 10^{-2})$

Table S2. Default simulated values of the hardware constraints and non-idealities in our study. These values are used to simulate the memristor-based recurrent neural network (RNN) with the IBM Analog Hardware Acceleration Kit [12].

relevant peripheral circuit at the behavioural level. We extend the original toolkit to consider the specific TiO_x -based resistive memory devices used in our work. The non-idealities we discuss in the hardware characterization section of the main manuscript are implemented in the simulator. For instance, the statistical programming variability is calibrated on our TiO_x -based resistive memory devices.

Re-training methods

For hardware-aware re-training, the main manuscript focuses on introducing a dropconnect training method during the re-training of the MND to mitigate the effect of defective devices in the chip. However, there are other training methods that could improve performance. One such method consists in the addition of noise on weights during forward passes of training steps to account for the programming variability of the characterized TiO_x -based resistive memory devices. It has been shown that setting the training weight variability approximately equal to the inference weight variability is optimal [13]. However, in our case, we observed training instability issues with this method, which led to an unexpected gradient explosion in a fraction of the trained RNNs. Therefore, this method did not benefit the overall performance.

The study of various non-idealities highlighted that only stuck-at fault devices have a critical effect on the performance of decoding. We attribute this to the fact that TiO_x -based resistive memory devices exhibit a low programming variability (in comparison to phase-change memory devices reported in [13]); therefore, injecting noise during training does not significantly improve fidelity. Also, an ADC and DAC of 8-bit resolution is sufficient for maintaining a fidelity close to the digital baseline, as explained in Supplementary Information Note S2. Thus, the input and output quantization during training does not allow for greater fidelity. Moreover, it is likely that the binary output of the model helps mitigate most noise-related issues since only a perturbation near the threshold value would impact the final binary output of RNNs.

Another limitation of memristors is their limited resistance range, which constrains the parameters’ values. This limitation can easily be tackled to improve test accuracy by using a method called weight clipping to trim the parameters’ values in a specific range during re-training [13]. It allows a certain degree of control over the weight distribution to make the programming of weights to hardware easier. It is performed by clipping the weights of a layer, after each backward pass in the training steps, within the range $[-\alpha\sigma; \alpha\sigma]$, where α represents the scale of the clipping, and σ is the standard deviation of the weight distribution of that layer. Based on our empirical tests, the exact value of σ does not appear to strongly affect the decoder’s performance. Thus, we decided to fix $\sigma = 2.5$ for all MND test simulations.

The decoding fidelity globally improves when the dropconnect method is introduced during re-training compared to the case where no dropconnect is employed (shown using a blue curve with the label “0%” in Fig. S4). It also appears that an RNN decoder trained with a high dropconnect rate will have a lower fidelity if no device is stuck, but its performance will decrease more slowly when the rate of stuck-at fault devices increases. Therefore, a good trade-off between fidelity and robustness seems to be a dropconnect rate equal to the number of blocked parameters (19 %) given the expected stuck-at fault device percentage (10 %). Since a parameter is encoded by two devices (for positive and negative values), the probability of a parameter being blocked (i.e., set to 0 %) is equal to the probability of at least one of the pair of devices being stuck.

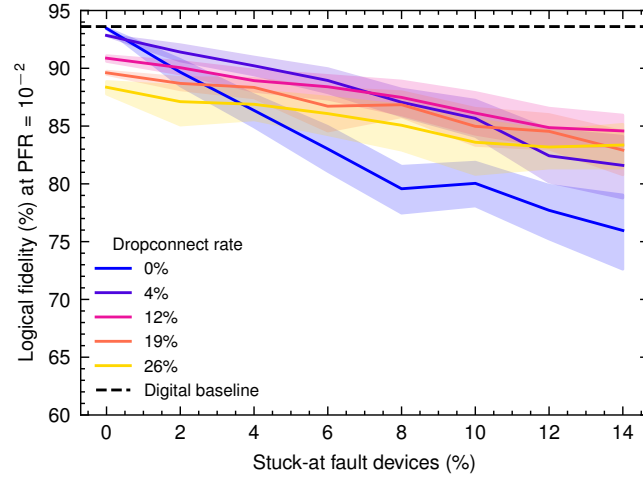


Figure S4. Impact of the dropconnect re-training method for various stuck-at-fault rates. Decoding fidelity at a physical fault rate (PFR) of 10^{-2} , for different dropconnect probabilities during hardware-aware (HWA) memristive neural decoder (MND) re-training, as a function of the percentage of stuck-at fault devices during inference. The shaded error regions represent the 95% confidence interval over 10 random seeds.

-
- [1] V. Yon and G. Dagnew, “A memristive neural decoder for cryogenic fault-tolerant quantum error correction - syndromes dataset,” (2024).
 - [2] C. Gidney and M. Ekerå, *Quantum* **5**, 433 (2021).
 - [3] O. Higgott, “Pymatching: A Python package for decoding quantum codes with minimum-weight perfect matching,” (2021), 2105.13082.
 - [4] C. Chamberland and P. Ronagh, *Quantum Science and Technology* **3**, 044002 (2018).
 - [5] M. Spear, J. E. Kim, C. H. Bennett, S. Agarwal, M. J. Marinella, and T. P. Xiao, *IEEE Journal on Exploratory Solid-State Computational Devices and Circuits* **9**, 176 (2023).
 - [6] C. Wang, H. Wu, B. Gao, T. Zhang, Y. Yang, and H. Qian, *Microelectronic Engineering* **187-188**, 121 (2018).
 - [7] P.-A. Mouny, Y. Beilliard, S. Graveline, M.-A. Roux, A. E. Mesoudy, R. Dawant, P. Gliech, S. Ecoffey, F. Alibart, M. Pioro-Ladrière, and D. Drouin, *IEEE Transactions on Electron Devices* **70**, 1989 (2023).
 - [8] F. Alibart, L. Gao, B. D. Hoskins, and D. B. Strukov, *Nanotechnology* **23**, 075201 (2012).
 - [9] Y. LeCun, Y. Bengio, and G. Hinton, *nature* **521**, 436 (2015).
 - [10] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, in *Advances in Neural Information Processing Systems 32* (Curran Associates, Inc., 2019) pp. 8024–8035.
 - [11] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” (2017), 1412.6980.
 - [12] M. J. Rasch, D. Moreda, T. Gokmen, M. Le Gallo, F. Carta, C. Goldberg, K. El Maghraoui, A. Sebastian, and V. Narayanan, in *2021 IEEE 3rd International Conference on Artificial Intelligence Circuits and Systems (AICAS)* (2021) pp. 1–4.
 - [13] V. Joshi, M. Le Gallo, S. Haefeli, I. Boybat, S. R. Nandakumar, C. Piveteau, M. Dazzi, B. Rajendran, A. Sebastian, and E. Eleftheriou, *Nature Communications* **11**, 2473 (2020).